Rapport intégral d'implémentation et d'audit : eGenBrain v4

Rapport technique reproductible

24 octobre 2026

Résumé exécutif

eGenBrain v4 prolonge le moteur de récupération v3 par une couche de maillage inférentiel typé : faits et occurrences persistants, primitives explicites, règles déclaratives à plusieurs prémisses, opérateurs déterministes typés, activation, clôture bornée et certificats vérifiables. La v3 reste disponible comme baseline et repli explicite. La réponse de calcul n'est acceptée comme réponse maillée que lorsque son certificat passe la vérification correspondante.

Les validations réalisées sont concrètes : 136 tests de régression réussis ; 21 tests ciblés de certification réussis ; 66 inférences aveugles sur onze micro-mondes et six modes, avec réponses brutes persistées avant le scoring ; intégration sur la base historique de 1 648 779 264 octets ; huit étapes API et six étapes CLI ; démarrage à froid sans base préexistante. Dans le laboratoire fini, le mode complet obtient 8/8 dérivations positives correctes et 3/3 abstentions attendues. Cette mesure est strictement limitée aux cas construits et ne constitue pas une preuve d'intelligence générale.

Chapitre 1

Description et objectifs

L'objectif est d'intégrer, dans le runtime v3 existant, la théorie de maillage formalisée dans `theorie_de_maillage_eGenBrain_0.tex`, tout en conservant l'héritage v2/v3. L'architecture générale distingue la géométrie observationnelle (empreintes et occurrences), la géométrie relationnelle (faits typés et relations dirigées), et la géométrie inférentielle (règles, opérateurs, transformations et preuves). Ces couches sont liées mais non identiques.

Le formalisme complet se trouve dans `formalime_eGenBrain_v4.tex` ; le registre `preuves_eGenBrain_v4.jso` relie les propositions aux lignes de code, aux tests et aux traces.

Chapitre 2

Architecture

2.1 Héritage

La classe `EGenBrainV4` étend `EGenBrainV3`. Elle conserve ASCR, la base MG historique et la possibilité de repli retrieval. La base historique n'a pas été réentraînée ni modifiée : SHA-256 `736a953b0350f1212bf87c2e46dceba1a68b0ff9797d5ec07214bb87526842ea`. Le nouveau maillage persistant est un sidecar SQLite séparé.

2.2 Composants ajoutés

— **MeshStore** : persistance des faits, types, occurrences, règles, opérateurs, activations, certificats et runs.
— **Activation** : sélectionner une région de faits sur les valeurs de la requête et étendre les dépendances de prédicats/règles ; les clés de schéma ne deviennent pas de faux ancrages.
— **Moteur de règles** : unification de variables, jointure de prémisses et clôture à point fixe avec limites de tours/faits.
— **Opérateurs** : addition, soustraction, multiplication et division via des signatures déterministes ; les compositions dimensionnelles vérifient les unités.
— **Certificats** : réexécution indépendante des opérations et vérification de prémisses, types, facteurs de conversion et provenance.
— **API/CLI/Studio** : ingestion, consultation, inférence et inspection des traces, y compris en démarrage à froid.

Chapitre 3

Pipeline d'exécution complet

1. Recevoir une question naturelle ou un motif structuré et calculer son empreinte.
2. Activer les faits et primitives pertinents ; conserver les identifiants et la politique d'activation.
3. Pour les requêtes structurées, rechercher le fait exact et/ou appliquer les règles déclarées.
4. Pour les tâches numériques prises en charge, extraire des observations via une grammaire bornée, trouver les primitives d'échelle et composer les opérateurs typés.
5. Construire un certificat détaillant les opérations, leurs entrées/sorties, les faits de primitive et les dimensions.
6. Recalculer les opérations et vérifier le certificat. Les facteurs ne sont pas acceptés parce que leur hash est cohérent seulement : ils doivent correspondre au fait primitif réellement stocké.
7. Si un résultat maillé certifié est disponible, le retourner sans repli retrieval. Pour une requête logique structurée non prouvée, s'abstenir. Pour les autres requêtes sans résultat maillé, exécuter le repli v3 et marquer explicitement que sa réponse n'est pas une preuve maillée.
8. Persister la réponse brute, statut, trace, certificat et temps. Le scoring est exécuté séparément après fermeture des inférences.

Chapitre 4

Commandes d'exécution

Depuis la racine du répertoire installé :

```
# Vérifier la base héritée
python -m egenbrain verify --model model/eGenBrain_v4.sqlite3 --full-check

# Démarrer avec le MG historique
python -m egenbrain serve --model model/eGenBrain_v4.sqlite3

# Démarrer dans un chemin vide sans restaurer le MG livré
python -m egenbrain serve --model model/eGenBrain_v4_fresh.sqlite3 --empty-start

# Ajouter des faits à un sidecar neuf
python -m egenbrain mesh4 fact --mesh-db model/eGenBrain_v4_mesh.sqlite3 \\
  --subject A --predicate edge --object-json '"B"'
python -m egenbrain mesh4 fact --mesh-db model/eGenBrain_v4_mesh.sqlite3 \\
  --subject B --predicate edge --object-json '"C"'
# Ajouter une règle depuis un fichier JSON, inférer et interroger
python -m egenbrain mesh4 rule --mesh-db model/eGenBrain_v4_mesh.sqlite3 --input rule.json
python -m egenbrain mesh4 infer --mesh-db model/eGenBrain_v4_mesh.sqlite3
python -m egenbrain mesh4 query --mesh-db model/eGenBrain_v4_mesh.sqlite3 \\
  --pattern-json '{"subject":"A","predicate":"edge","object":"C"}'

# Chat v4; maillage certifié puis repli v3
python -m egenbrain chat --model model/eGenBrain_v4.sqlite3 --task chat \\
  --message 'Tim buys 3 dozen eggs. Eggs cost $.50 each. How much did he pay for eggs?'
```

Les commandes du laboratoire (exécutées en développement) sont documentées dans `Amodel` ; elles ne doivent pas être confondues avec les commandes runtime ci-dessus.

Chapitre 5

Résultats expérimentaux

5.1 Suite logicielle

La suite complète s'est terminée avec **136 tests réussis sur 136** en 17,828 secondes, avec `ResourceWarning` configuré comme erreur. La suite de certification ciblée contient 21 tests, dont un test adverse modifiant le facteur d'unité et recalculant le hash du certificat ; ce certificat forgé est rejeté au motif que le facteur ne correspond pas à la primitive persistante.

5.2 Campagne comparative aveugle

La campagne utilise 11 cas x 6 modes = 66 inférences. Le runner n'importe pas le fichier des réponses de référence. Il enregistre chaque réponse brute et son empreinte avant le lancement de l'outil de scoring. Le contrôle post-hoc a vérifié les 66 hashes, la matrice complète, 55 bases SQLite du laboratoire et 25 contrôles de cibles structurées, sans fuite de fait cible observé détectée.

Mode	Résultat exact/abstention	Positifs / 8	G
Retrieval v3 seul	3/11	0/8	0,000
Observations seules	3/11	0/8	0,000
Primitives sans règles/opérateurs	3/11	0/8	0,000
Primitives + opérateurs	6/11	3/8	0,375
Primitives + règles	8/11	5/8	0,625
Maillage complet	11/11	8/8	1,000

G représente ici la fraction des huit cas positifs qui requièrent une composition et sont correctement dérivés. Dans le mode complet, les onze résultats incluent trois abstentions correctes. La campagne est finie et conçue autour de primitives/règles explicites ; ses scores ne doivent pas être extrapolés à la langue ou au raisonnement général.

5.3 Résultat des œufs et transfert aux poires

La question des œufs retourne 18 avec statut `MESH_DERIVED`. Le plan utilise la primitive `dozen -> 12`, la quantité 3 et le prix 0,50 extraits de l'entrée, puis deux multiplications. Le certificat est vérifié. Le test des poires demande deux douzaines à 1,25 par poire et retourne 30, avec la même forme de composition mais une entité et une quantité/prix différentes. Aucun enregistrement de réponse cible n'est utilisé comme fait observé.

La portée doit rester précise : le facteur de douzaine est explicitement fourni et la multiplication est un opérateur général codé, pas un opérateur découvert à partir des exemples. Le parseur naturel prend en charge une grammaire bornée et n'est pas un parseur de langue générale.

5.4 Intégration du MG et fallback

La base MG de 1 648 779 264 octets garde l'empreinte attendue ; SQLite `integrity_check` retourne `ok`, sans violation de clé étrangère. En intégration v4, la question des œufs donne 18 avec certificat `PASS` et sans fallback. Une requête distincte sur la capitale de la France traverse le fallback v3, qui répond `NO_ASCR_RESULTS` ; l'abstention est conservée.

5.5 API, CLI et démarrage à froid

Le test HTTP réussit 8 étapes de bout en bout, y compris l'accès à l'interface web, la création de faits/règle, la clôture, la requête du fait dérivé et le chat certifié. Le test CLI réussit six étapes avec codes retour zéro. Le mode `-empty-start` démarre sans base existante, ne crée pas de données d'entraînement synthétiques, construit le sidecar et permet d'importer faits/règles. Les deux bases SQLite créées passent l'intégrité et les clés étrangères.

Chapitre 6

Interprétation et conséquences

6.1 Conséquences théoriques

Le résultat est compatible avec une interprétation du maillage comme structure calculable, et non seulement comme index de retrieval, lorsque l'on ajoute des règles et des opérateurs exécutables. Mais les résultats ne démontrent pas que la géométrie de maillage suffit à expliquer l'intelligence. Les relations observationnelles, les déductions explicites et les opérations arithmétiques doivent rester distinguées.

6.2 Conséquences conceptuelles

Une trajectoire dans un graphe n'est pas automatiquement une preuve ; la clôture logique nécessite des règles explicites et les calculs dimensionnels nécessitent opérateurs/types/primitives. La v4 introduit donc un statut distinct `MESH_DERIVED`, un certificat et un validateur.

6.3 Conséquences opérationnelles

L'utilisateur peut créer le maillage depuis des faits/règles nouveaux, consulter les traces, utiliser le mode vide et conserver la base historique. Les réponses brutes sont persistées avant le scoring. Le mécanisme de repli v3 est marqué comme retrieval, jamais comme certificat de raisonnement maillé.

Chapitre 7

Questions ouvertes et limites

— Étendre et évaluer le parsing de langue naturelle sur paraphrases tenues à l'écart, ambiguïtés et langues différentes.
— Démontrer l'induction de règles non déclarées et le transfert vers des structures nouvelles, plutôt que seulement l'exécution de règles explicites.
— Quantifier précision, fausses dérivations, couverture, abstention, latence et coût sur des corpus nettement plus grands.
— Tester propriétés DDD/empreintes à plusieurs résolutions contre des baselines simples et ablations.
— Valider les conflits, négation, non-monotonie et règles de priorité sur une batterie spécifique.

Ces points sont ouverts et testables ; ils ne sont pas transformés en verdict d'impossibilité.

Chapitre 8

Statut du modèle

Domaine	Statut fondé sur les artefacts
Maillage typé persistant	Implémenté et testé
Règles déclaratives multi-prémisses	Implémentées et testées sur les règles déclarées
Opérateurs arithmétiques typés	Implémentés en tant que primitives génériques
Certificats de dérivation	Vérificateur implémenté ; test adversarial de facteur d'unité réussi
Intégration historique v3	Test d'intégration sur vrai MG réussi
Démarrage sans base	Test HTTP/CLI réel réussi
Composition sur micro-monde fermé	8/8 cas positifs dans le mode complet, limitée aux cas construits
Compréhension générale du langage	Non démontrée ; grammaire de quantité/coût bornée
Induction autonome de règles	Ouverte ; les règles actuelles sont déclaratives
Intelligence générale	Non mesurée par ces expériences

Chapitre 9

Traçabilité et artefacts

Les principales preuves sont : `logs/full_test_suite_v4_pass1.txt`, `logs/lab_leakage_audit.json`, `lab_results/lab_raw_results.jsonl`, `lab_results/lab_scored_report.json`, `logs/real_v4_integration_` `logs/v4_http_empty_start_api_final.json` et `logs/v4_cli_mesh_final.json`. Le fichier `preuves_eGenBrain` contient la correspondance des concepts, sources, lignes, tests, états et empreintes SHA-256.

Chapitre 10

Installation propre et validation des archives de runtime

Les deux archives d'exécution ont passé le contrôle CRC et le contrôle des noms de membre. Le ZIP principal compte 178 000 318 octets ; le ZIP complémentaire compte 199 698 071 octets, donc aucun des deux ne dépasse 220 MiB. Le réassembleur a été lancé à partir des archives finales, dans un répertoire d'installation neuf ; il a reconstitué le MG historique à 1 648 779 264 octets. L'empreinte obtenue, `736a953b0350f1212bf87c2e46dceba1a68b0ff9797d5ec07214bb87526842ea`, correspond au manifeste original ; `integrity_check=ok` et zéro violation de clé étrangère.

10.1 Inférence réelle depuis le runtime installé

Depuis le dossier extrait, sans charger le workspace de développement, la requête en anglais *Tim buys 3 dozen eggs. Eggs cost $0.50 each. How much did he pay for eggs ?* a retourné `18` avec le statut `MESH_DERIVED`, deux étapes de multiplication typée, un certificat vérifié et `retrieval_fallback_executed=false`. La primitive `dozen=12` était incluse dans la base de primitives ; le parseur a extrait les quantités de l'entrée et la multiplication est un opérateur général pré-déclaré. Le test n'est donc ni une réponse stockée, ni une démonstration d'induction autonome d'opérateur.

10.2 API depuis le runtime installé en mode vide

Un second processus serveur démarré sur un chemin SQLite inexistant a répondu aux endpoints de santé, page, JavaScript, insertion de faits, insertion de règle, inférence jusqu'au point fixe, requête du fait dérivé et chat certifié (8 étapes). Les deux bases créées par ce parcours ont passé l'intégrité SQLite et les vérifications de clés étrangères, et le MG installé est resté inchangé. C'est une validation de l'API et du bootstrap sans données artificielles ; ce n'est pas une mesure de performance statistique.